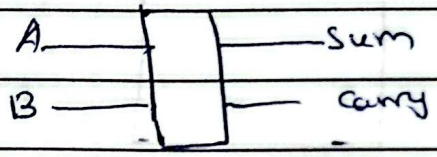
/date

Digital Building Blocks (Adders)

First of all let's see how adder works:

Half Adder:



A	B	Sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

so

$$\text{Sum} = A \oplus B$$

$$\text{Carry} = A \cdot B$$

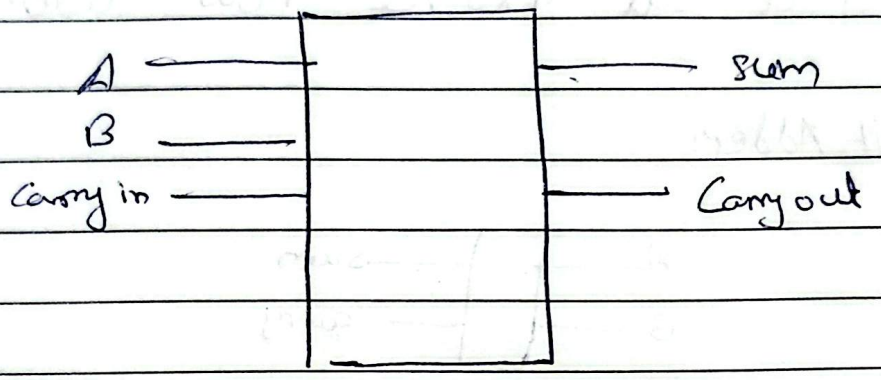
BHATTI

CamScanner

day/date

Full Adder:

Half adder has a problem, that it does not deal with carry in. so full adder fixes that.



A	B	Carry in	Sum	Carry out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

BHATTI

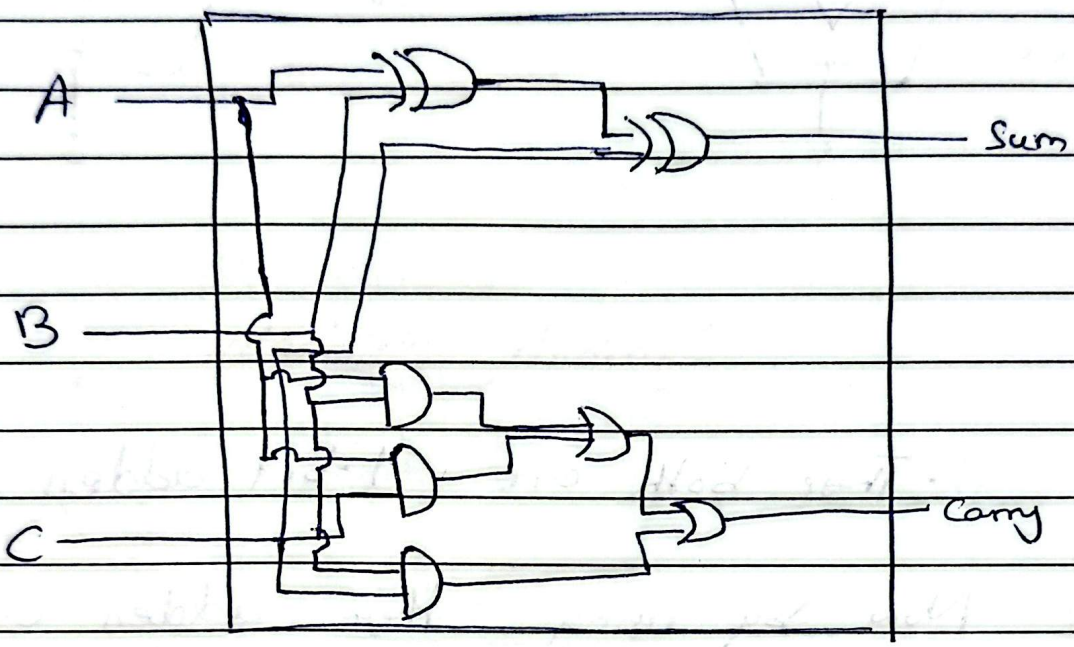
CamScanner

y/date

So

Sum = $A \oplus B \oplus$ Carryout

Carry = $A \cdot C + B \cdot C + A \cdot B$



Full adder

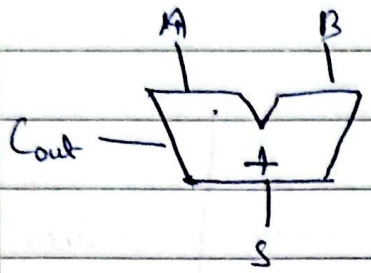
BHATTI

CamScanner

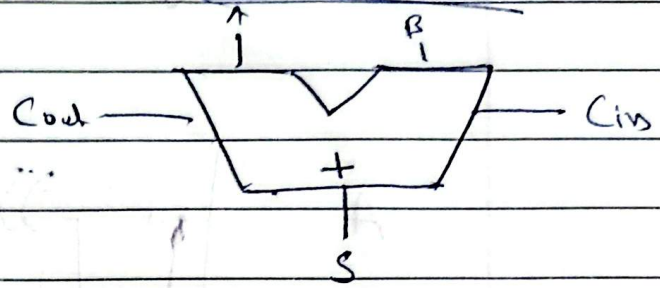
y/date

Both:

Half adder:



Full adder.



Symbols for both.

- These both are 1-bit adders.

Now by using these adders we can create bigger adders: e.g 32 bit.

Multibit adders:

1) Ripple carry (slow)
2) Carry-lookahead (fast)
3) Prefix (faster)

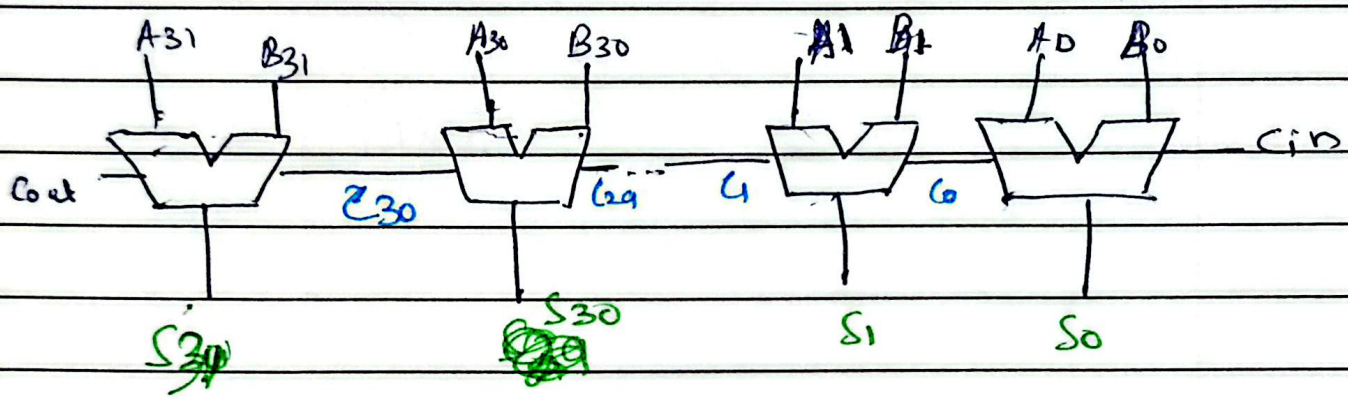
BHATTI

Ripple Carry Adder:

In this we chain 1-bit adders together & carry generated are rippled through entire chain.

- It is slow.

32-bit adder:



The problem in this is that every need to wait for the adder before it to produce a carry.

BHATTI

CamScanner

day/date

$$t_{ripple} = \overset{Total}{delay} = N t_{FA}.$$

N = number of 1 bit adders.

t_{FA} = delay of a full adder.

BHATTI

CamScanner

date

Carry - Lookahead Adder:

We have seen that the ripple is very slow.
This look ahead Adder was made to make it faster. Increasing speed leads to more hardware but speed increases a lot.

The whole idea of Carry-lookahead is to see the previous bits and if carry exists add in the particular 1-bit adder.
Now carry will be decided before any F.A runs.

We know that every 2 bits can either generate a carry or Propogate a carry.

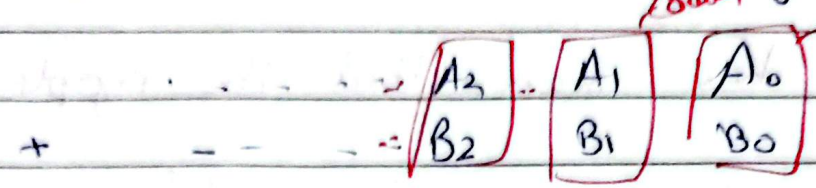
So simply we calculate the generation (g_i) & propagation (P_i) signals for each column.

BHATTI

CamScanner

/date

By column I am referring to:



Generation:

A column i will generate a carry out if Ai and Bi both are 1

$$G_i = A_i B_i$$

A = 1
B = 1
Cout = 1 → carry generated.

Propogate:

A colum i will propogate a carry in to the carry out if either of A or B are 1.

$$P_i = A_i + B_i$$

BHATTI

CamScanner

te

Together:

$$C_i = A_i B_i + (A_i + B_i) C_{i-1}$$
$$= G_i + P_i C_{i-1}$$

Examples:

C_3 C_2 G_1 G_0

0 1 1 0

1 0 1 1

+ 1 0 0 1

~~Carry.~~

~~0~~

P_3	P_2	P_1	P_0
1	0	1	1

G_3	G_2	G_1	G_0
1	0	0	1

so using this we get.

C_3 C_2 C_1 C_0

1 1 1 1

carry in

1 1 1 1 0

how we can parallel running the adder

0 1 1 0

1 0 0 1

0 0 0 1

BHATTI

//date

Block of Propagate & generate:

C_0, C_1, C_2, C_3 all can be formed into equation using current bit & org Carry in

$$C_0 = G_0 + P_0 C_{-1}$$

$C_{-1} = 0$ in addition

$$C_0 = A_0 B_0 + (A_0 + B_0) C_{in}$$

$$C_1 = A_1 B_1 + P_1 C_{i-1}$$

$$= A_1 B_1 + P_1 C_0$$

$$C_1 = A_1 B_1 + P_1 (A_0 B_0 + (A_0 + B_0) C_{in})$$

& so on

By this we can see any carry ahead can be generated using the inputs & the original carry in (which is also an input).

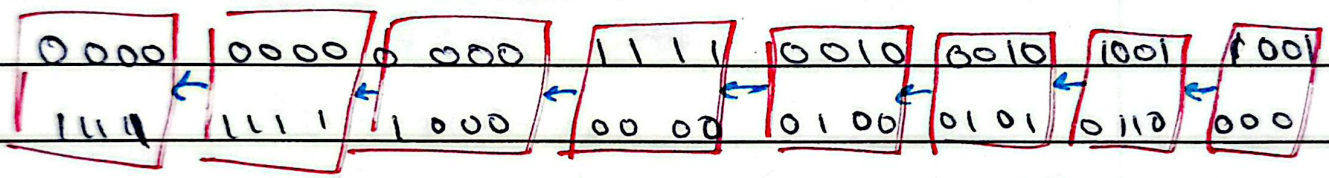
BHATTI

:e

Block propogate & Generate:

In theory If we generate carry for all 32 bit, that would adding very fast but practically would cause a lot of hardware Over head hence we divide our 32 bits or as many bits into k-bits. block. & to apply carry look ahead on those blocks.

e.g 4 bit blocks:



- Now look ahead carry generator produces carry in for all blocks.

ite

- Now we will use column Propogate & generate singnals to compute Block Propogate & Block generate.

- Now we compute if a k-bit group will propogate a carry in of block to out of block or not.

&

- Compute if a k-bit group generates a carry or not.

Block Propogate signal:

Let's take 4 bit blocks.

so

we need to find

$P_3 P_2 P_1 P_0 = ?$

$P_{3:0} = P_3 P_2 P_1 P_0 = ?$

easier representation

BHATTI

e.g

A = 1011

B = 0001

& carry into this 4 bit block is 1

```
  0 1 11
  1 0 11
+ 0 0 01
-------
  1 1 01
```

$P_0 = 1 + 1 = 1$

$P_1 = 1 + 0 = 1$

$P_2 = 0 + 0 = 0$

$P_3 = 0 + 1 = 1$

$P_i = A_i + B_i$

$P_{3:0} = P_3 \cdot P_2 \cdot P_1 \cdot P_0 = 1 \cdot 1 \cdot 0 \cdot 1 = 0$

so no propogation.

BHATTI

date

Block generate:

Now for generate: lets again take 4 bit block:

$$G_{3:0} = ?$$

for generation if our G_0 generates that that also needs to be propagated to carry out by C_1, C_2, C_3 so

If G_0 then $P_1 P_2 P_3$ are must

If G_1 happens then $P_2 P_3$ are must

If G_2 happens then P_3 is must

If G_3 happen carry out.

So

$$G_{3:0} = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_3 P_2 P_1.$$

$$G_{3:0} = G_3 + P_3 [G_2 + P_2 (G_1 + P_1 G_0)]$$

BHATTI

CamScanner

ate

Example:

carry out ← 0 0 1 1 ← carry in $C_3, C_2, C_1, C_0, C_{-1}$

$A_{3:0}$: 1 0 0 1

$B_{3:0}$: + 1 1 0 0

$S_{3:0}$: 0 1 1 0

$P_0 = 1 + 0 = 1$	$G_0 = 1 \cdot 0 = 0$
$P_1 = 0 + 0 = 0$	$G_1 = 0 \cdot 0 = 0$
$P_2 = 0 + 1 = 1$	$G_2 = 0 \cdot 1 = 0$
$P_3 = 1 + 1 = 0$	$G_3 = 1 \cdot 1 = 1$

$P_i = A_i + B_i$

$G_i = A_i \cdot B_i$

So $G_{3:0} = 1 + \dots$

$= 1$

BHATTI

date

Block Propogate & generate:

Together if we see them we get

$$C_3 = G_{3:0} + P_{3:0} C_{-1}$$

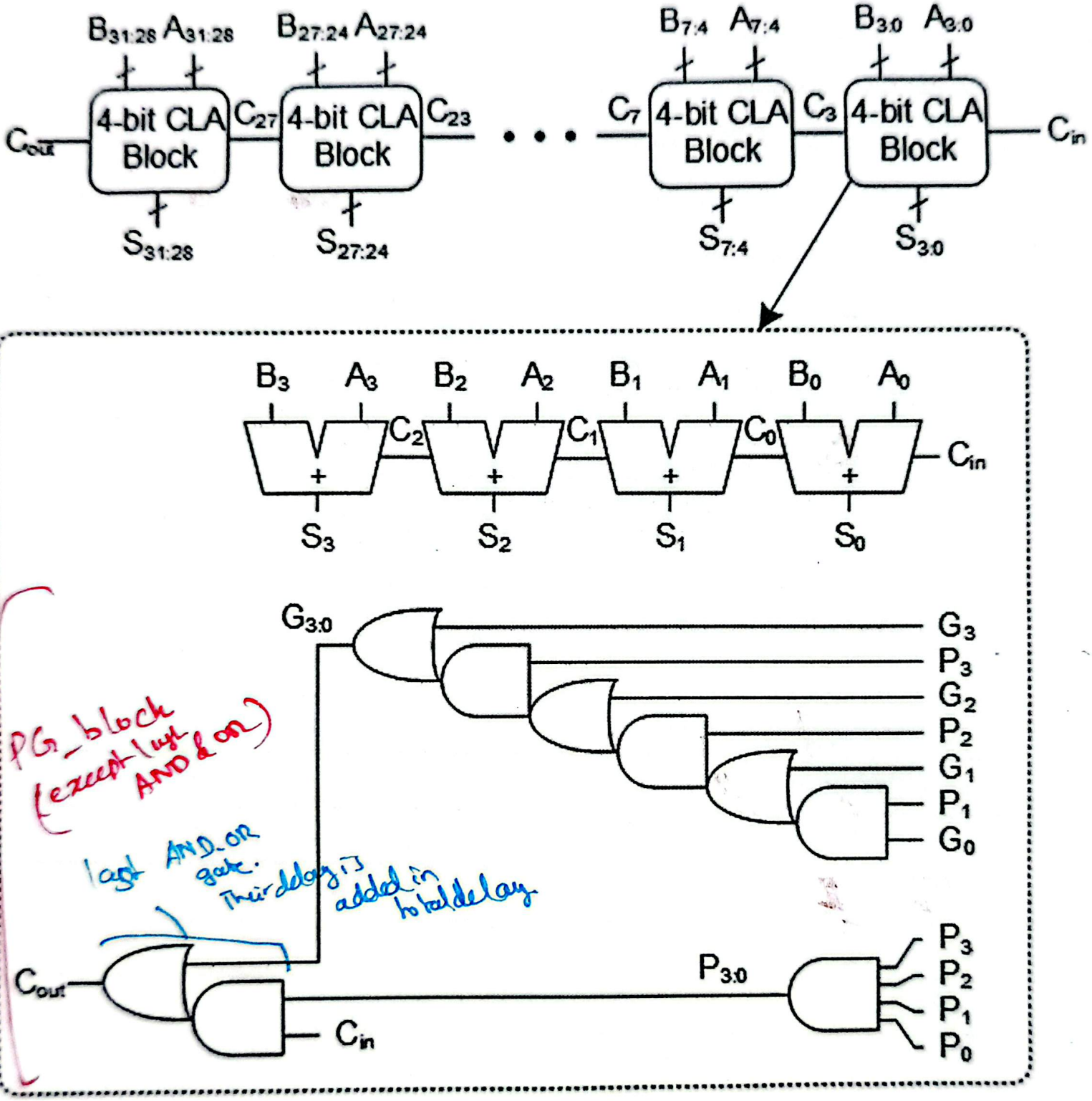
$P_{i:j}$ & $G_{i:j}$ can easily be computed using only A & B inputs because they are only dependent on it.
& thus with only only one carry in that is to the block we can find carry out.

so we can see how we can find any ahead carry using just the starting carry in.

BHATTI

CamScanner

Diagram:



PG_block (except last AND & or)

last AND-OR gate. Their delay is added in total delay

Every 4-bit block has both the propagator/generator & the adders for summing.

BHATTI

y/date

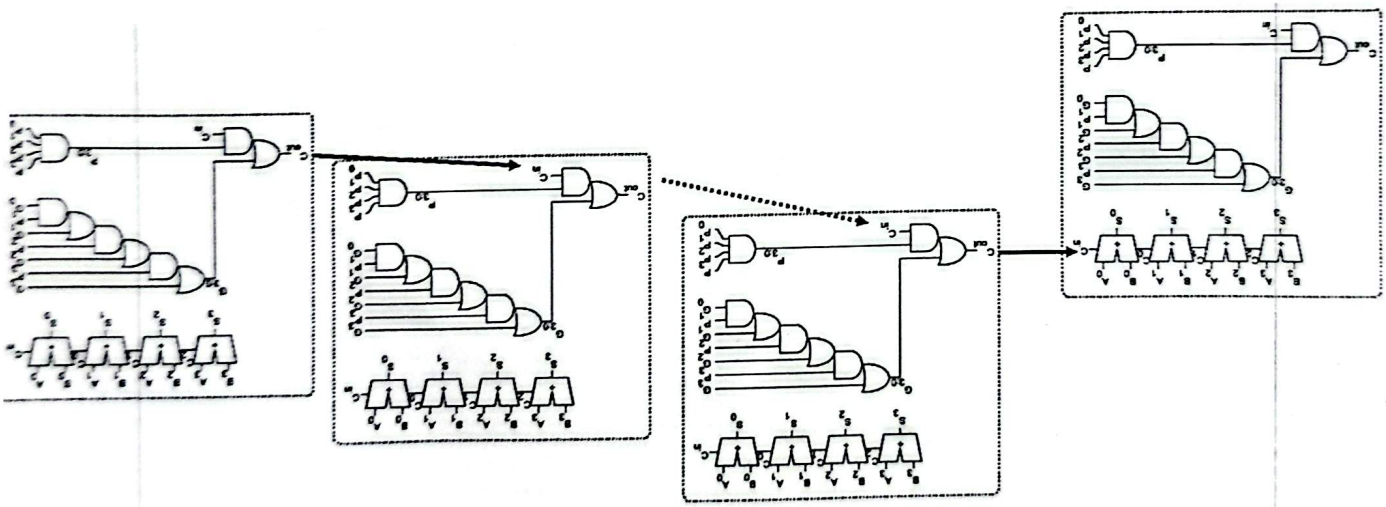
Carry look ahead addition:

Step1) Compute G_i & P_i for all columns

Step2) Compute G & P for k-bit blocks

Step3) C_{in} propagates through each k-bit propagate / generate logic (meanwhile Sums computed)

Step4) Compute Sum for most significant k-bit block



BHATTI

CamScanner

/date

Carry look ahead (CLA) adder delay:

$$t_{CLA} = t_{pg} + t_{pg_block} + \left(\frac{N}{k} - 1\right) t_{AND_OR} + k t_{FA}.$$

t_{pg} = delay to generate all P_i & G_i

t_{pg_block} = delay to generate all $P_{i:j}$, $G_{i:j}$

t_{AND_OR} = delay from C_{in} to Cout of final AND/OR in k-bit CLA block.

In the diagram on 2nd last page we can see that every block uses ripple Carry for inside it but uses between blocks look ahead it used.

So like a mixture of both.

BHATTI

CamScanner

date

Example Question:

Compare delay of 32 bit, ripple carry & CLA

CLA has 4 bit blocks, a full adder delay is 300 ps. A single gate delay is 100 ps.

N = 32

$t_{ripple} = 32 * t_{FA} = 32 * 300$

$= 9.6$ ns.

$t_{CKA} = t_{pg} + t_{pg\,block} + \left(\frac{32}{4} - 1\right) t_{AND_OR} + k t_{FA}$

t_{pg}: P & G both cost 1 gate each. = 100 ps

$t_{pg\,block}$: A block has 6 gates (7 actually but one works in parallel) so 6 total gates: = 600 ns.

$\left(\frac{32}{4} - 1\right)$: 7

t_{AND_OR}: First AND then OR so 2 gates 200 ns.

$k t_{FA}$: $t_{FA} = 300$, $k = 4$. All FA blocks work in parallel.

$= 100 + 600 + 7 * 200 + 300 * 4$

$= 3.3$ ns.

BHATTI

CamScanner

ate

So we got almost 3X the speed by CLA.

BHATTI

CamScanner

/date

Digital Building Blocks. (Subtraction & comparators)

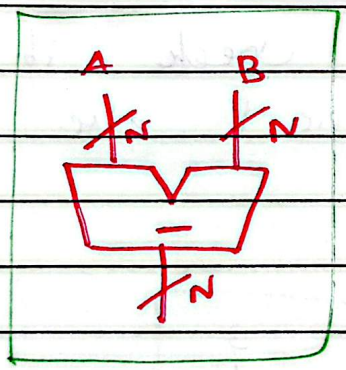
Subtractor:

It uses the logic:

$$A - B = A + \overline{B} + 1$$

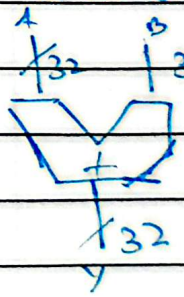
Basically it uses an adder with B input flipped & carry in (C_{-1}) = 1.

Symbol:



N = number of bits.

Instead we can also write a constant

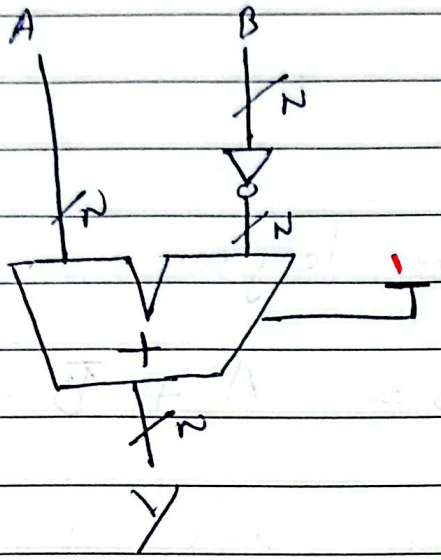


BHATTI

CamScanner

ay/date

Implementation:

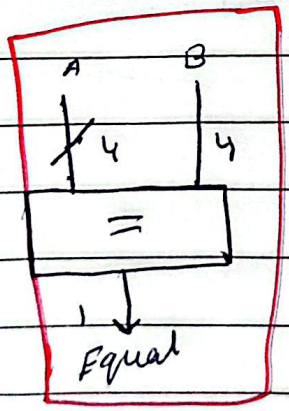


using adder actually.

Equality Comparator:

It this we check if A == B, if equal give 1 else 0.

symbol:

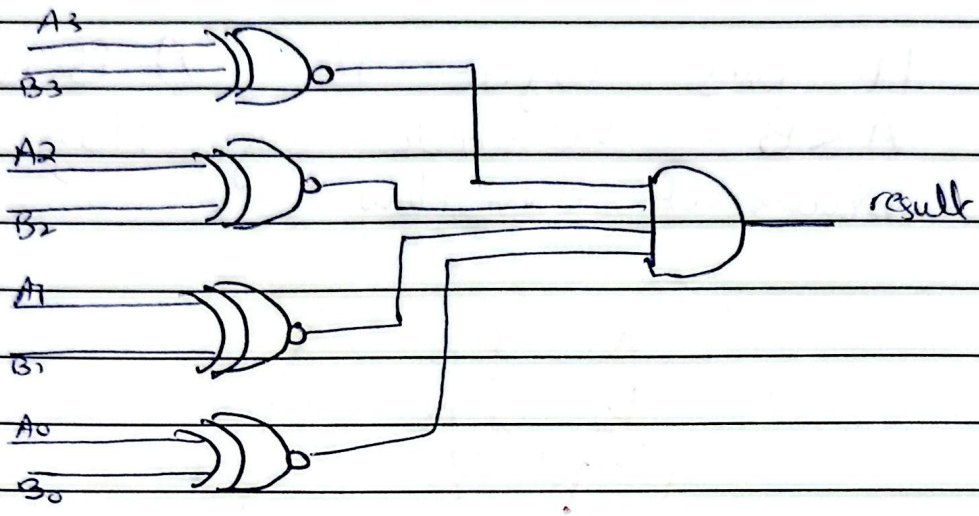


BHATTI

CamScanner

date

Implementation for 4bits



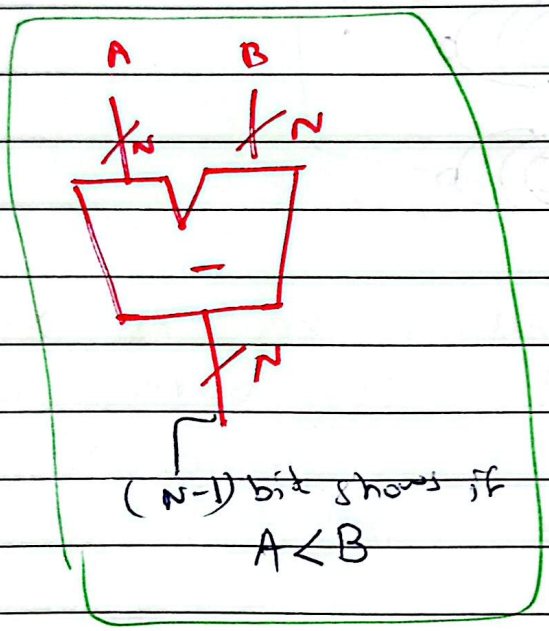
* XNOR is used as XNOR gives 1 if both bits are same.

BHATTI

'date

Signed less than comparator.

It uses logic if A-B is negative A < B. we check sign flags & remain aware of over flow



BHATTI

CamScanner

ay/date

Digital Building Blocks (ALU)

An basic ALU should perform:

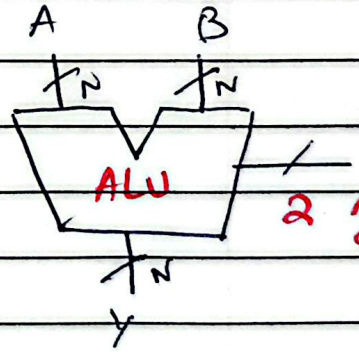
* Addition
* Subtraction
* AND
* OR.

But we should be able to select which operation.

As there are 4 operations so 2 bits are needed.

These are called ALU control bits.

ALU control 1:0	Function
0 0	Add
0 1	Subtract
1 0	AND
1 1	OR.



BHATTI

CamScanner

ıte

e.g A = 1011 , B = 0101

ALU control = 10

So

= A • B

Answer = 0001

How to differentiate addition & Subtraction?

0 0	Add
0 1	Subtract

we will use the ALU control bit number 0 to check if we need to add or subtract.

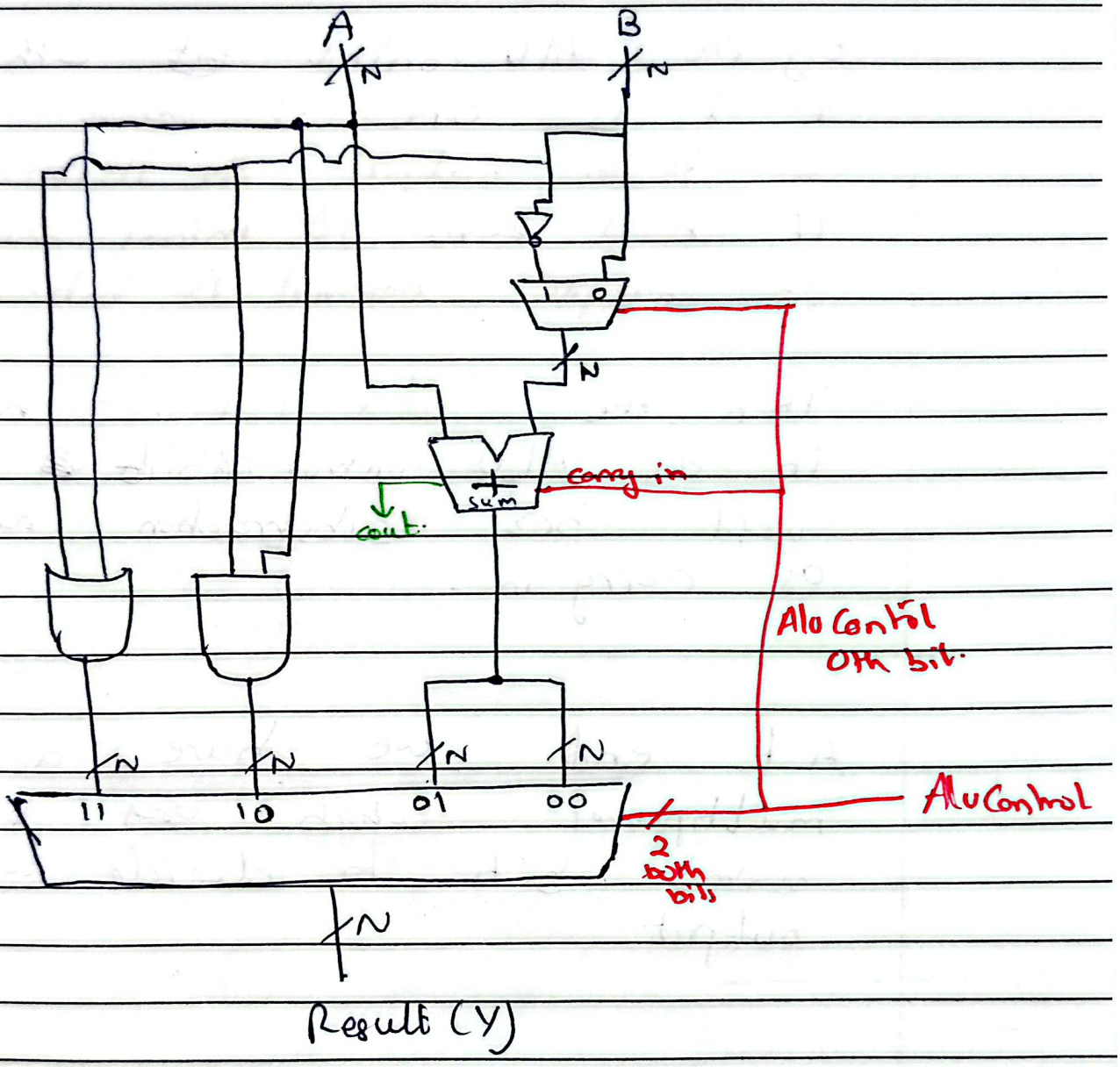
BHATTI

CamScanner

ate

ALU inside:

Diagram covering these 4:



BHATTI

CamScanner

ate

Now if we see the diagram the top multiplexer (mux) [1 0] is controlled by the ALU control 0th bit
If it is one, it means subtraction so output the inverted B bit.
If it is zero it means addition so output normal B bit.

Then the Adder Circuit has carry in of ALU control 0th bit as well bcz subtraction need 1 as carry in.

At end we have a 4:1 multiplexer which uses both ALU control bits to decide what to output.

BHATTI

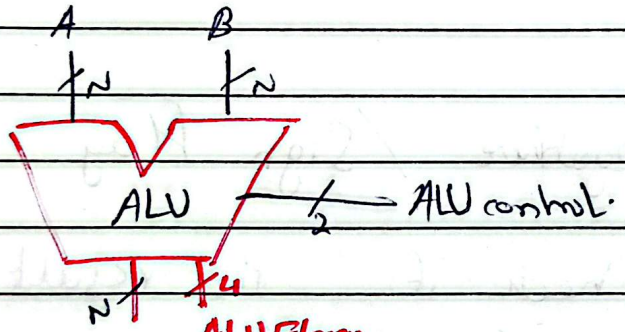
CamScanner

ate

Status Flags:

But an ALU doesn't end with these 4 operations it also has status flags:

- N : Negative → 1 if result is negative
- Z : zero → 1 if result is 0
- C : Carry → 1 if carry out produced
- V : overflow → 1 if adder overflowed.



1) Zero flag:

If the output of ALU is exactly zero then Z = 1

If output not zero then Z = 0.

BHATTI

ate

To find zero flag result we simply need to check if all bits are zero.

So we use NOR.

A 32 input NOR gate applied across all bits of ALU Result.

$$Z = \overline{ALUResult_{31} + ALUResult_{30} + \cdots + ALUResult_0}$$

2) Negative / Sign Flag (N)

Check if the result of operation is negative

It is simply the last 31st bit of the Result

$$N = ALUResult\ 31$$

BHATTI

late

3) Carry Flag (C)

Captures the final Carry bit produced out of the top bit (bit 31) of 32 bit adder.

It is used to check Unsigned over flow detection & multiword addition

$$C = C_{32} = Cout.$$

In Subtraction if $C = 1$, it shows no borrow. ($A \geq B$ unsigned).
but if $C = 0$, it shows borrow happened.
($A < B$ unsigned)

But we will also keep it 0 during AND & OR so

$$C = C_{32} \cdot \overline{ALU\ Control_1}$$

BHATTI

date

4) Overflow flag (V)

Indicates that a signed arithmetic operation produces a result that exceeded the 32-bit 2's complement (-2^{31} to $(2^{31}-1)$)

Also here it should only turn on if needed during ADD & SUB never during OR & AND ~~&~~.

$$V = \overline{ALU\,Control\,0 \oplus A_{31} \oplus B_{31}} \cdot A_{31} \oplus Sum_{31} \cdot \overline{ALU\,control\,1}$$

The first: $\overline{ALU_{control}\,0 \oplus A_{31} \oplus B_{31}}$ checks if operand A & B have same sign

~~&~~ Also ~~&~~ in subtraction B_{31} is flipped so to replicate that ALUcontrol 0 is also XNORed ~~that~~ with these.

Second: $A_{31} \oplus Sum_{31}$ checks if both have different bit. If different give a 1.

BHATTI

r/date

Third: ALU control
= so that only gives output on ADD or SUB, never on AND or OR.

we know why the third condition needed but why the first 2?

when can over flow happen?

A		B	
positive	+	positive	can happen
negative	+	negative	can happen
positive	+	negative	X never happen

Also remember ALU always adds so even if

= 7 - (-1)

write it at as

7 + 1

A ↑ ↑ B

so V can be 1.

so don't make decisions in hurry.

BHATTI

ate

so always ~~consider~~ make a Sum sign in between & then check both inputs.

ALU Control$_0$	A_{31}	B_{31}	XNOR of all 3	
0	0	0	1	overflow possible
0	0	1	0	~~not possible~~
0	1	0	0	
0	1	1	1	overflow possible
1	0	0	0	never V
1	0	1	1	overflow possible
1	1	0	1	
1	1	1	0	never V

By this first XNOR of 3 we find if overflow is possible or not & if our second condition that A_{31} & Sum_{31} have different signs/bits then that tells truly.

BHATTI

CamScanner

late

Sometimes we also get $A_{31} \oplus Sum_{31}$ as 1 but our $A_{31} \oplus B_{31} \oplus ALUControl0$ is 0 so then again no overflow.

Only if all 3 conditions match

Examples:

(Now I will considering 4 bit ALU)

1) $00^{A}11 + 11^{B}00$, so ALU control = 00

$$\begin{array}{r} 00\ 11 \\ +\ 11\ 00 \\ \hline \text{Sum} = 11\ 11 \end{array}$$

$= A_3 \oplus B_3 \oplus ALU\ control\ 0$

$= 0 \oplus 1 \oplus 0$

$= 0$ as just one got 0 so all AND result get 0

V = 0 , no overflow.

BHATTI

CamScanner

/date

0011 = 3

1100 = -4

3 -4

= -1 = 1111

no overflow as

4 bit range = -8 to 7

Hence verified.

2) A B

1000 | - 0010 , ALU control = 01

0010 → flipped → 1101 → +1 → 1110

2's complement

Sum 1001

Cout (1) + 1110

0111

BHATTI

y/date

First condition:

$= \overline{A_3 \oplus B_3 \oplus \text{Alu Control 0}}$

$= 1 \oplus 0 \oplus 1$

$= 1$

2nd condition:

$= A_3 \oplus Sum_3$

$= 1 \oplus 0$

$= 1$

$V = 1 \cdot 1 \cdot \overline{\text{Alu Control 0}}$

$= 1 \cdot 1 \cdot 1$

$= 1$

so overflow.

Verify:

$1001 = -7$

$0010 = 2$

$-7 - 2 = -9$

$-9 > -8$ so Overflow

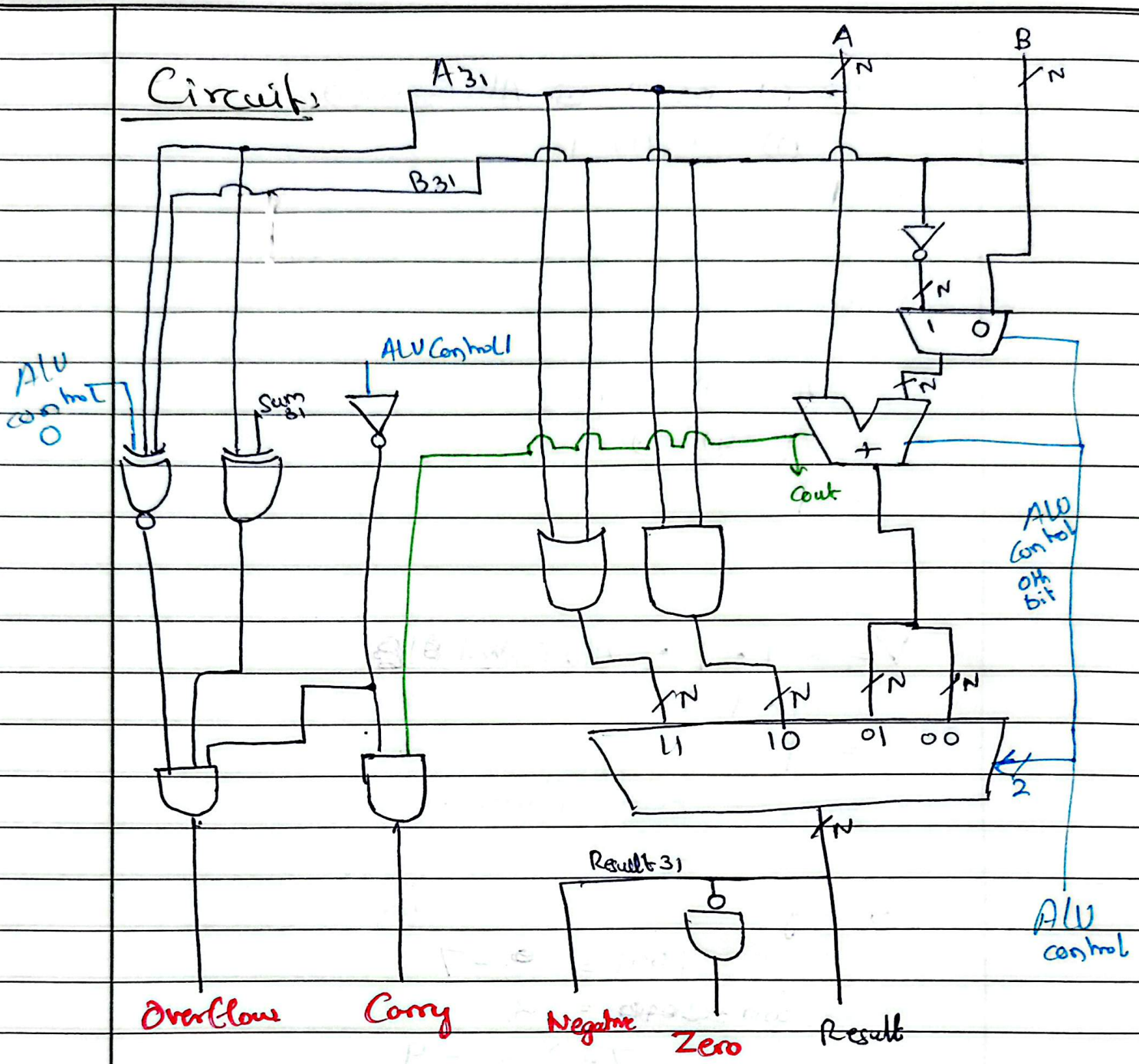
Hence verified.

BHATTI

CamScanner

day/date

Circuit:



* Instead of A31 or B31 it is actually the MSB but I am considering 32 bit ALU. for 4-bit it will be A3, B3, Sum3.

BHATTI

CamScanner

y/date

Comparison based on Flags:

Comparison	Signed	Unsigned
$=$	Z	Z
$\neq$	$\overline{Z}$	$\overline{Z}$
$<$	$N \oplus V$	$\overline{C}$
$\geq$	$\overline{N \oplus V}$	C
$\leq$	$Z + (N \oplus V)$	$Z + \overline{C}$
$>$	$\overline{Z + (N \oplus V)}$ $= \overline{Z} \cdot \overline{N \oplus V}$	$\overline{Z + \overline{C}}$ $= \overline{Z} \cdot C$

BHATTI

CamScanner

ay/date

Extending ALU: SLT

Before we said ALU only has 4 operation but it can actually have more but then our ALU control bit will also increase.

Now we will add SLT operation as well.

ALU Control 2:0	Function
000	Add
001	Sub
010	and
011	OR
101	SLT

* 100 is for XOR, basically just like and, OR.

BHATTI

CamScanner

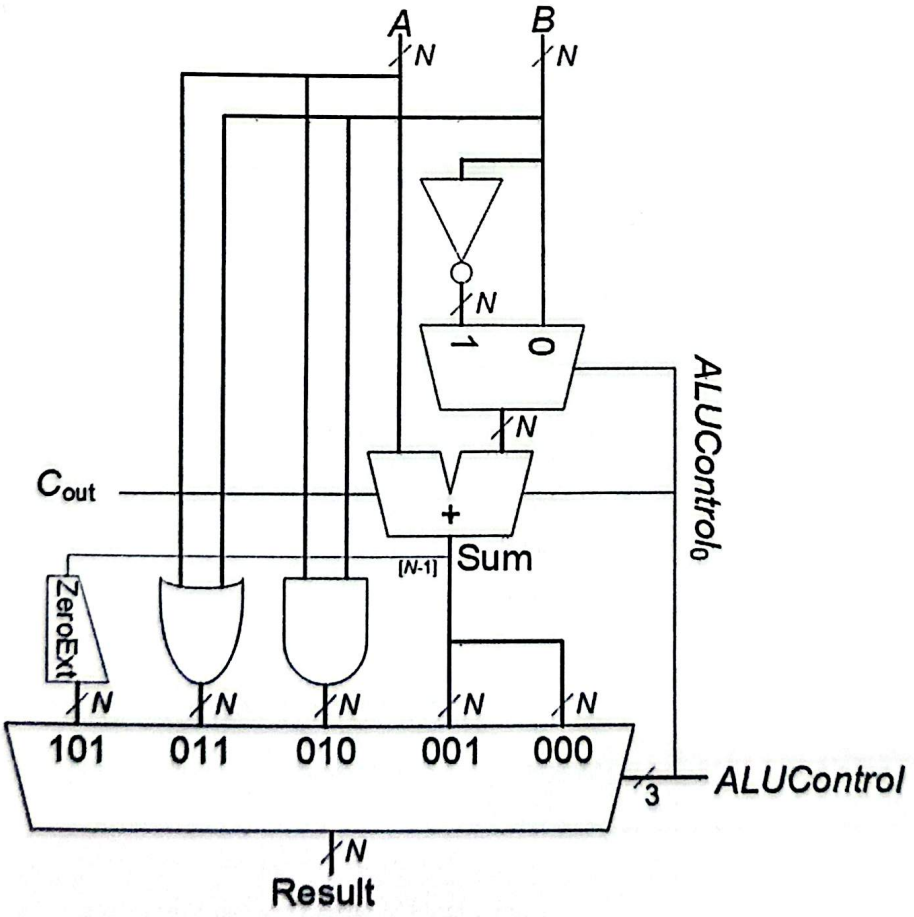
ate

Now how to find SLT?

we can just use the Sum_{31} bit !!

If negative then A < B but if positive then A ≥ B.
So Sum_{31} solves it.

But SLT returns a 32 bit number not one bits,
So use a zero extender after it.



BHATTI

Now our this logic will work for many equation but for some it will give wrong results.

This happens in cases of signed Overflow.

e.g A = 0110 (+6) , B = 1011 (-5).

= ~~A - B~~ .

now check if ~~S~~ A ≤ B.

If yes then SLT = 1

but +6 > -5 so SLT should be 0.

Let's see using our current logic.

```
  1
  0 1 1 0
+ 0 1 0 1
-------
  1 0 1 1
```

now Sum_{3x} = 1

so -ve flag is set

BHATTI

e

so SLT = 1

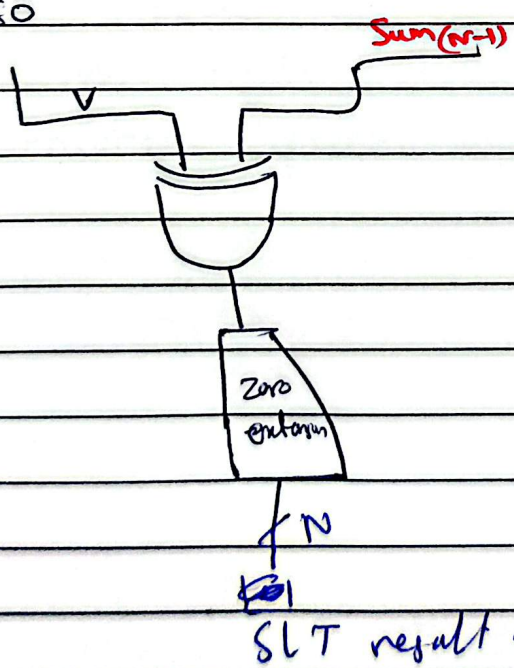
which is wrong!!

So what logic should we use to fix it?

basically , we know it happen when overflow happens so we ALWAYS get wrong results in overflow.

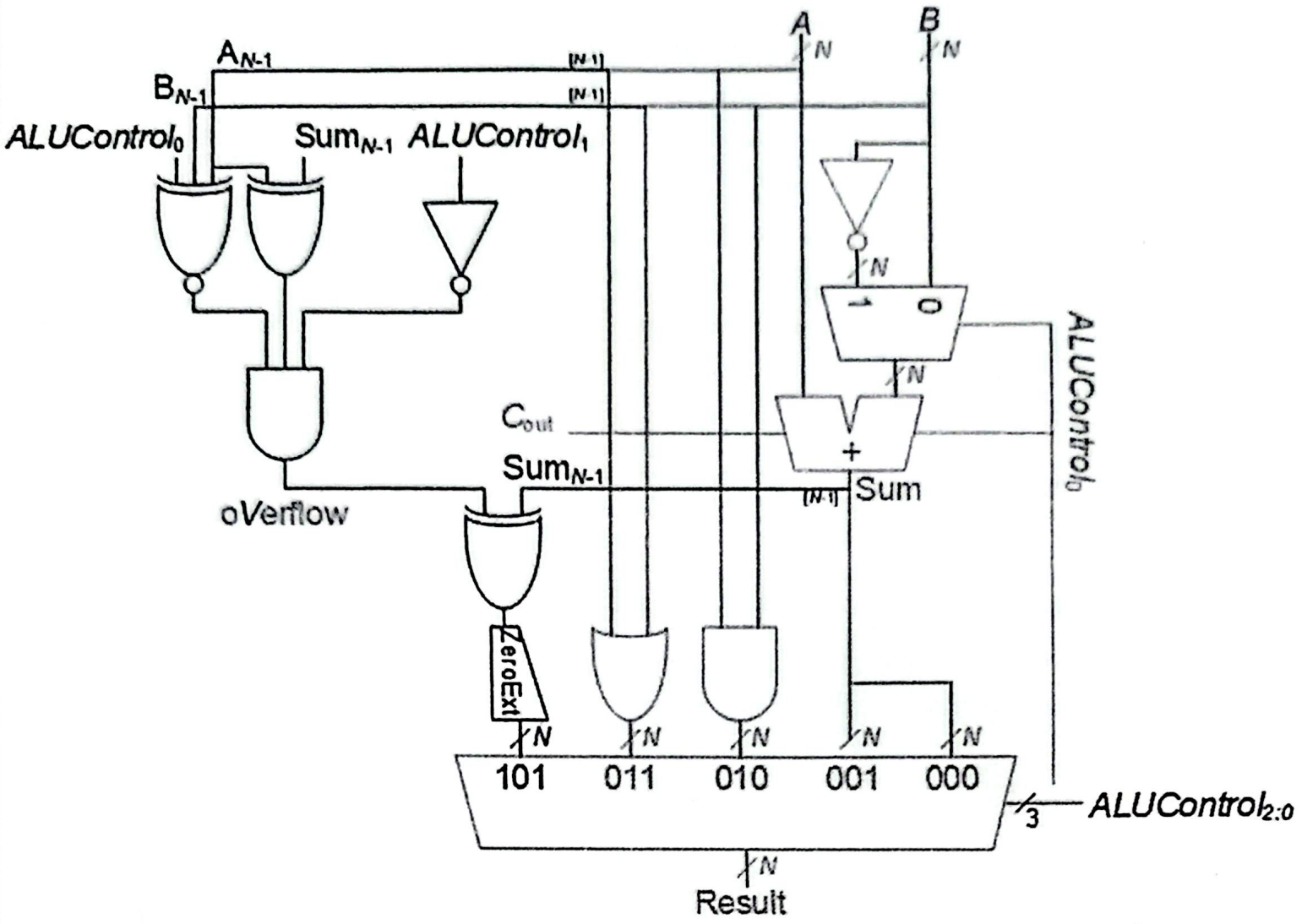
so simply invert the result of SLT if overflow

so



This fixes the problem.

BHATTI


A
B
N
A_{N-1}
B_{N-1}
[N-1]
$ALUControl_0$
Sum_{N-1}
$ALUControl_1$
1
0
C_{out}
Sum_{N-1}
+
Sum
oVerflow
ZeroExt
101
011
010
001
000
3
$ALUControl_{2:0}$
Result

late

Digital Building Blocks.
(Shifters, Multiplier & Dividers)

Shifters:

Logical Shifter:

shifts value left or right & fills empty spaces with 0's.

e.g $11001 \gg 2 = 00110$

$11001 << 2 = 00100$

symbol : "<<" or ">>"

Arithmetic Shifter:

Same as logical shifter but on right shift fills the right shift fills empty spaces with old MSB.

e.g

$11001 >>> 2 = 11110$

$11001 <<< 2 = 00100$

symbol : ">>>" or "<<<"

BHATTI

CamScanner

Rotate:

Rotates bits in a circle such that bits shifted off one end are shifted into the other end.

ROR = rotate right

ROL = rotate left.

e.g

11001 ROR 2 = 01110

11001 ROL 2 = 00111

BHATTI

CamScanner

Shift left Design: (logical)

How now do we design it in circuit.

We can use a Mux to decide for each ~~bit~~ result bit that either one of the iput bit will come or ground (0) bit comes.

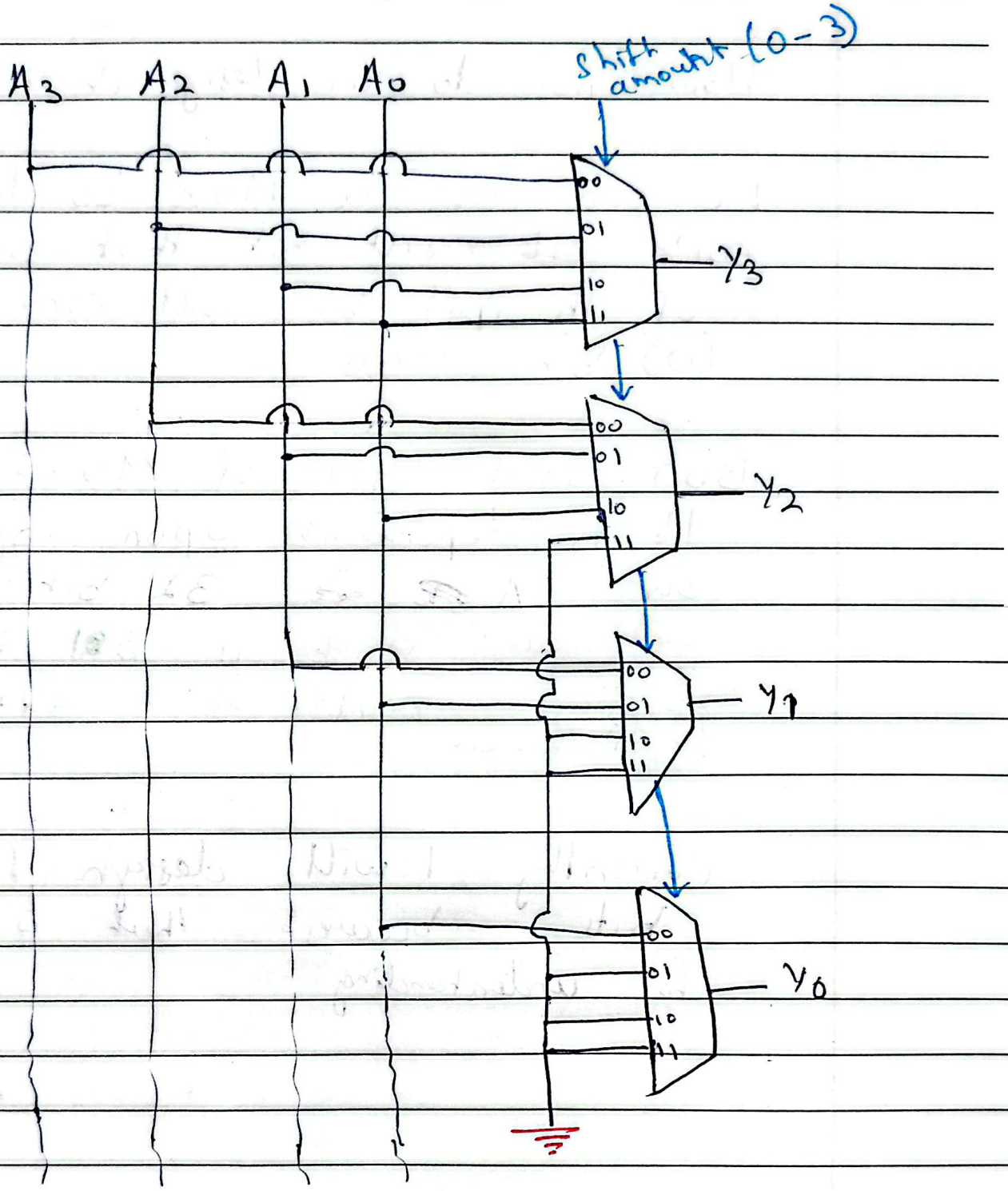
BUT what type of MUX?
It is dependent upon input if our A is 32 bit, the max we can shift is 31 so 32 decisions possible so 32:1 MUX.
(0-31) = 32 decisions.
shift.

Currently I will design for 4 bits because that is enough for understanding.

BHATTI

late

consider a 4 bit input A. so 4:1 Mux



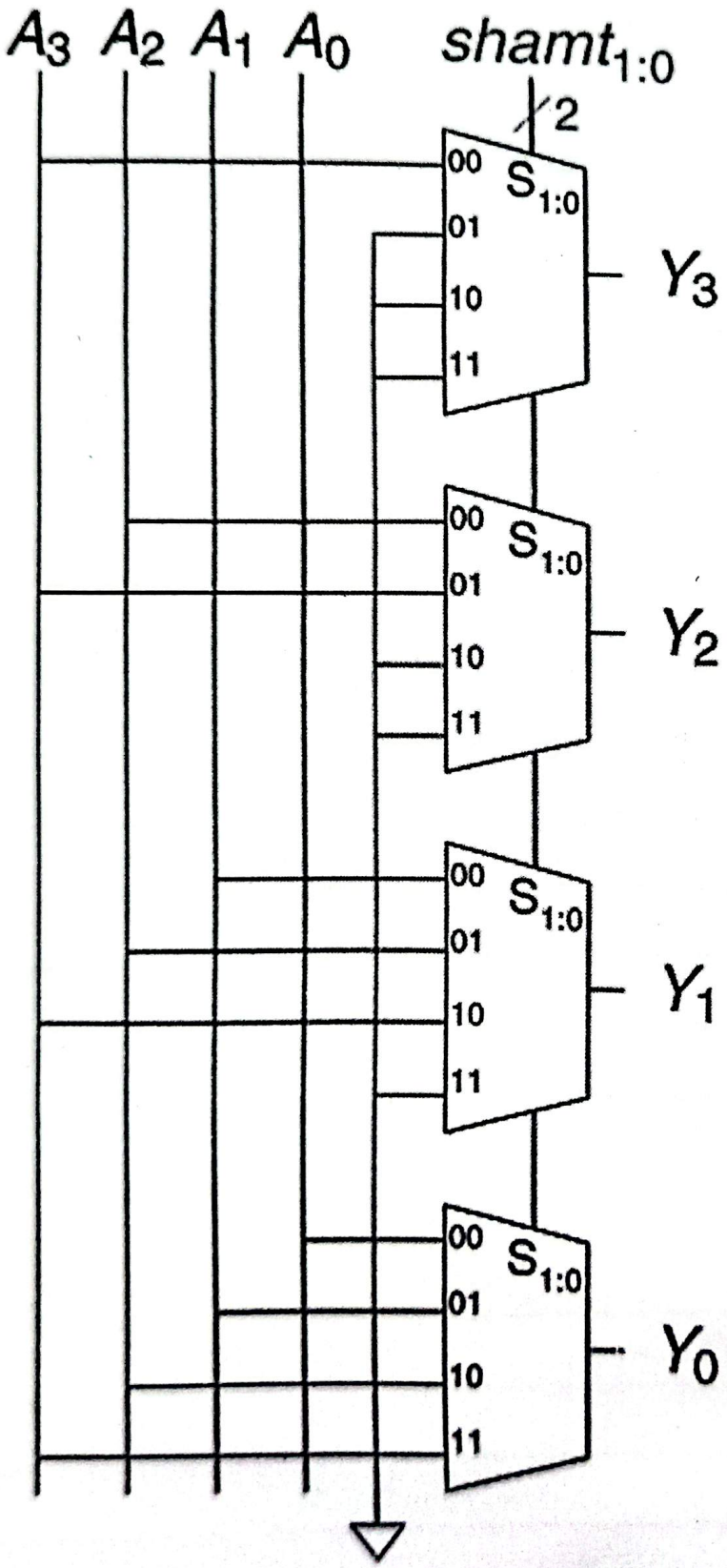
we can only shift 0, 1, 2, or 3 times. After this in Risc-V we typically should get error unless compiler handles it.

BHATTI

CamScanner

Logical Right shift design:

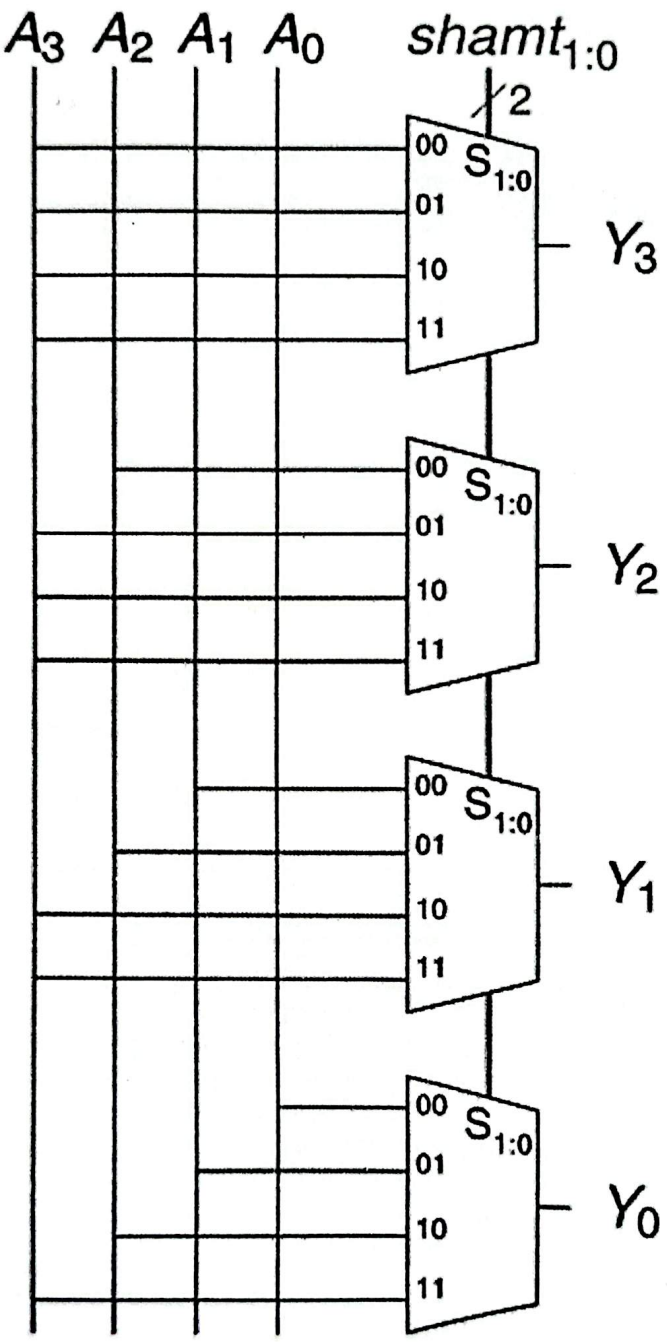
Just like the left logical shift one but basically like upside down.



BHATTI

CamScanner

Arthemetic Shift Right:
It is like shift logical Right but now instead of ground simply apply A_3 in all places where there was ground.

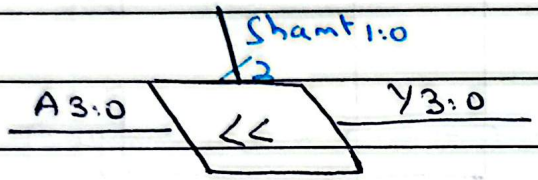


BHATTI

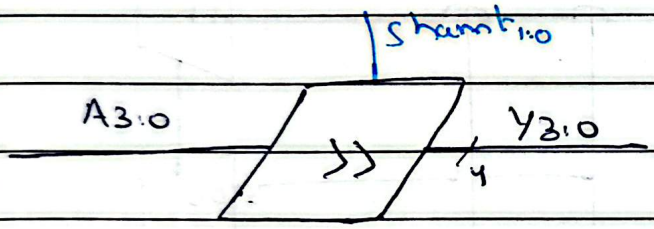
CamScanner

* Arithmetic left shift is exactly like logical left shift so I don't need to draw it.

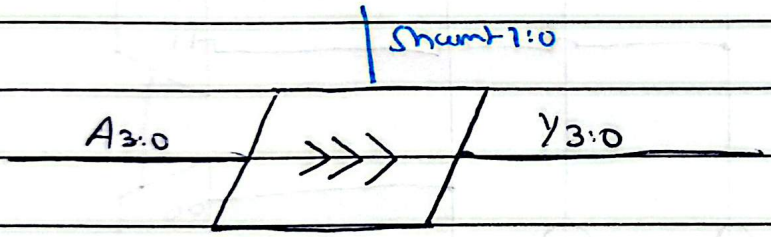
All 3 symbols:



Shift left logical / Arithmetic



Shift right logical



Shift right Arithmetic.

* These are for 4 bits.

Shamt = shift amount

BHATTI

'date

Rotate ~~Register~~ left.

Now instead of ground we strat to bring the lost bits back in.

BHATTI

late

* Try to make for ROR ourself.

BHATTI

CamScanner

date

Multiplies & Divides using Shifts.

We can multiply & divide in powers of 2 using shifter.

$$A << N = A * 2^n.$$

$$A >>> N \equiv A \div 2^n$$

BHATTI

CamScanner

date

Multiplier:

If we see normal multiplication.

```
    ①
    2 3 0
  x   4 2
  ---------
  ② 4 6 0      } Partial products
+ 9 2 0 x
  ---------
  9 6 6 0      ← sum of Partial products.
```

Binary:

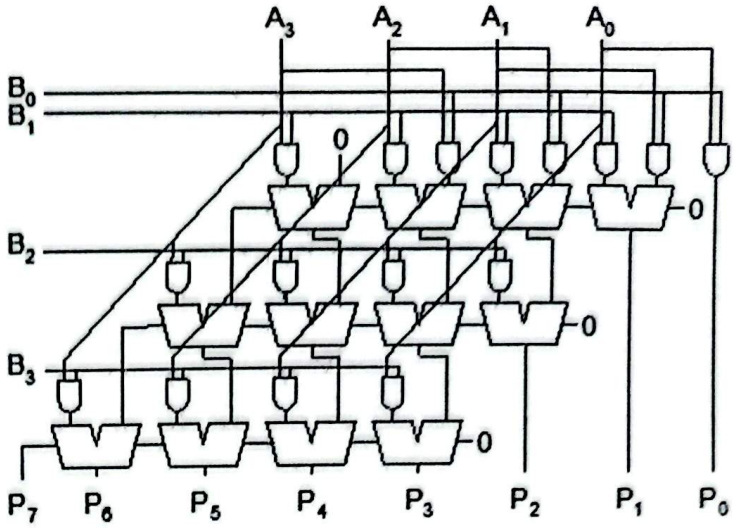
```
      0 1 0 1
    x 0 1 1 1
  ---------
    ① 0 1 0 1
  ① 0 1 0 1 x      } Partial Products
    0 1 0 1 x x
  0 0 0 0 x x x
  ---------
  0 1 0 0 0 1 1    ← sum of partial products
```

Now using the logic to make partial products & then adding them is what we will use to make a Multiplier.

BHATTI

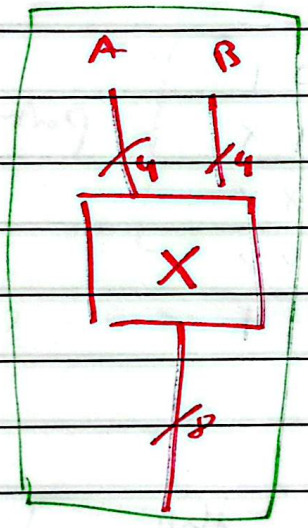
day/date

		A_3	A_2	A_1	A_0			
	x	B_3	B_2	B_1	B_0			
		A_3B_0	A_2B_0	A_1B_0	A_0B_0			
	A_3B_1	A_2B_1	A_1B_1	A_0B_1				
	A_3B_2	A_2B_2	A_1B_2	A_0B_2				
+	A_3B_3	A_2B_3	A_1B_3	A_0B_3				
P_7	P_6	P_5	P_4	P_3	P_2	P_1	P_0	



Simply used AND & full adders.

Symbol



Also see products are always 2* bits of input.

BHATTI

CamScanner

'date

Dividers:

First see how we divide the long-hand way:

2584 /15

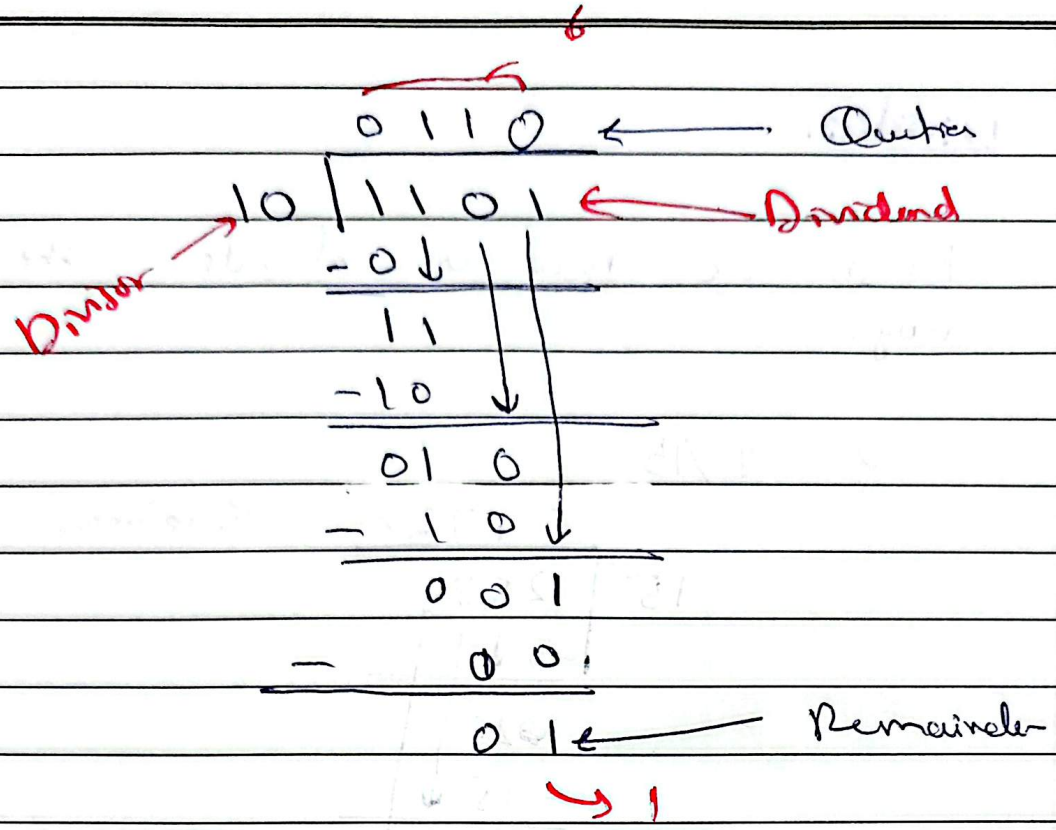
```
      0172   <- Quotient
   ┌──────
15 | 2584
   -0 ↓ |
    025 |
   -15 ↓
    108
   -105 ↓
     34
   -  30
   ------
      04   <- Remainder
```

Now in binary do:

1101 /10 ≡ 13/2 = 6 & 1
(remainder)

BHATTI

CamScanner



Now for making the machine we can see the Logic:

- Shifts the Dividend & store into a buffer.
- Subtract Divisor from it & store in buffer.
- If result is negative then it means buffer was currently smaller so bring buffer back by adding the DVSor & make $Q_i = 0$.

If result is positive, make Q_i 1. keep buffer as is.

- Repeat this process until all bit of Dividend have been brought to buffer

BHATTI

CamScanner

date

Logic in code

Dividend → $\frac{A}{B} = Q + \frac{R}{B}$ ← remainder

Divisor → B

Quotient → Q

R = Temprevry remainder (buffer basically)

R' = Temparory shifter remainder with next dividend bit A_i Appended.

```
R' = 0
for i = N-1 to 0
    R' = R' << 1

    R' = R' + Ai
    R = R'

    D = R - B.

    if D < 0
            Qi = 0 ; R' = R # bring back
    else
            Qi = 1 ; R' = D. # new left over.
R = R'
```

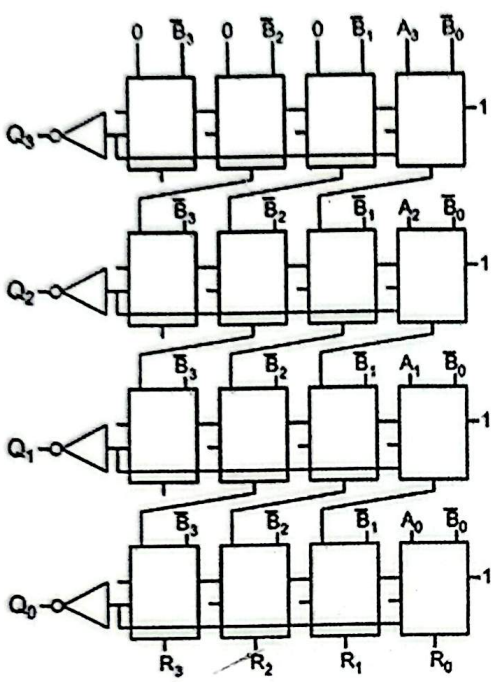
BHATTI

CamScanner

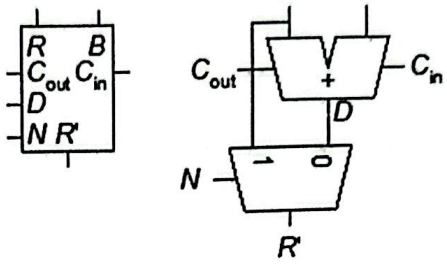
ay/date

Design:

Basically we are adding & then seeing if we want to keep that addition or not.



Legend



In this design we need to keep all the 4 bits in mind while most will be zero which is why above we can see 4 parts in each layer.

BHATTI

CamScanner